

Event-Driven Architecture

Exercise

Outline

In this exercise, we will begin with an existing application that implements the order cycle of an e-commerce company. The goal is to transform a monolithic, tightly-coupled Order Management workflow into a more efficient, scalable, and loosely-coupled architecture. We will achieve this by splitting key functionalities into separate, specialized modules that communicate asynchronously via Events and synchronously via Service Actions.

At the end of this exercise, we will have a blueprint of the entire process and the flow of events that need to happen to support the order cycle, as well as the necessary apps in ODC Studio to support it.

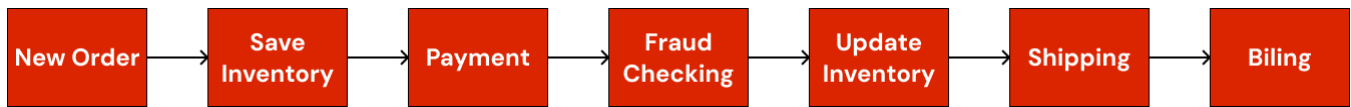
Use Case

Let's start by analyzing the use case.

Customer Portal

The EDA company is a very successful e-commerce company. They are always at the forefront of compliance as mandated by the local and international regulatory bodies, with very detailed verifications of every transaction.

They follow the standard order cycle as shown in the picture below, whereby an order goes through multiple stages before being shipped and billed. Most of the steps occur sequentially, making it difficult to scale as the server threads are kept waiting until the entire process completes.



In summary, when a new order is created, the process starts by checking the inventory and saving the products and quantity in the order. Then, there's the payment and verification to check for any fraud. If everything is ok, the inventory is updated, the order is shipped, and the customer is billed.

Due to the development process of this application over the years, implementing new changes or features can take months. This delay is unacceptable, especially given the potential vulnerabilities and the financial losses it could cause. Also, due to current trends in the online marketplace, more stringent checks need to be implemented and executed within hours or days. However, this is not feasible with the current app architecture.

During peak times, the infrastructure costs skyrocket due to millions of transactions that are being handled in that window, and the application is not able to keep up due to the sequential processing of requests.

For these reasons, EDA management is planning to revamp the entire app to make it flexible, scalable, and performant.

Quickstart App

This exercise begins with the EDA Store app, which is not fully implemented. Some functionalities have been replaced by empty Actions to simulate the real-world interactions.

The app has four Screens:

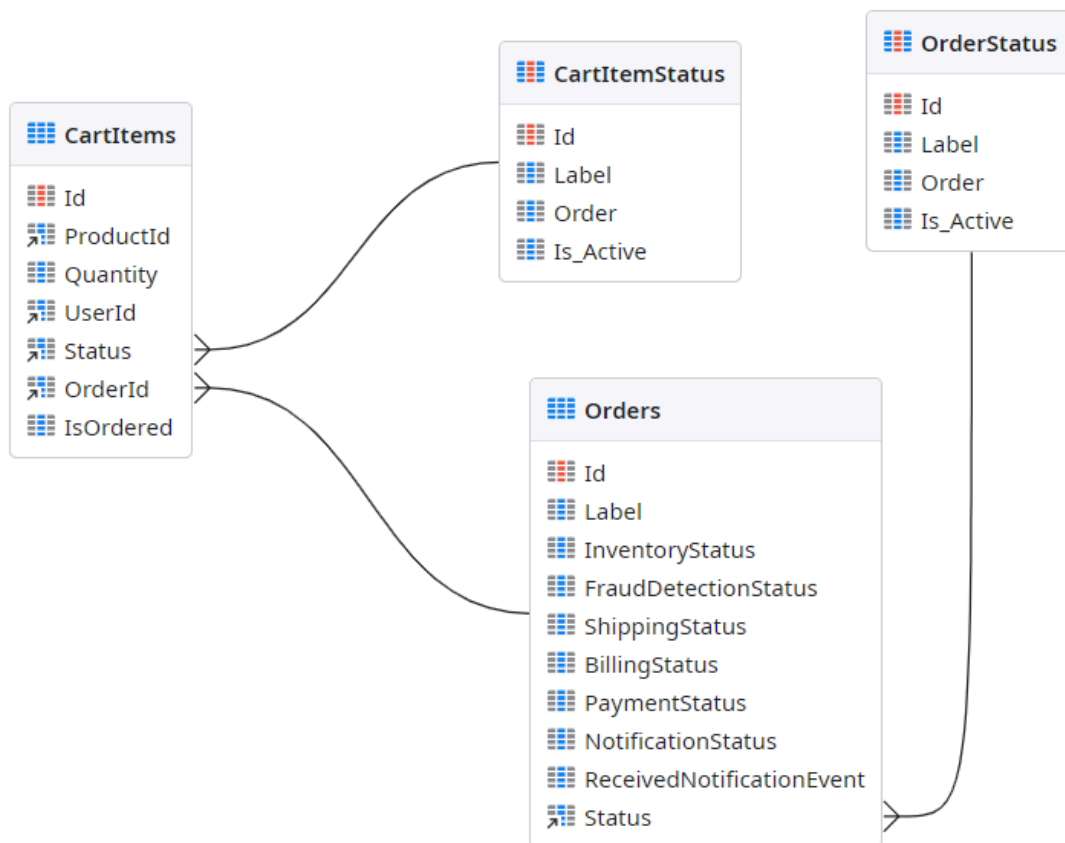
- **ProductCatalog:** The app's Home Screen. This Screen contains the product catalog and allows the user to select the product they want to buy.
- **ProductDetail:** The Screen displays the details of a Product and allows users to add products to the cart.
- **CheckoutCart, CheckoutComplete:** These Screens display the checkout cart UI, allowing users to finalize their purchase. The *CheckoutCart* Screen displays the list

of products in the cart and is accessible via the app's menu. The *CheckoutComplete* Screen displays the summary of the order that was purchased.

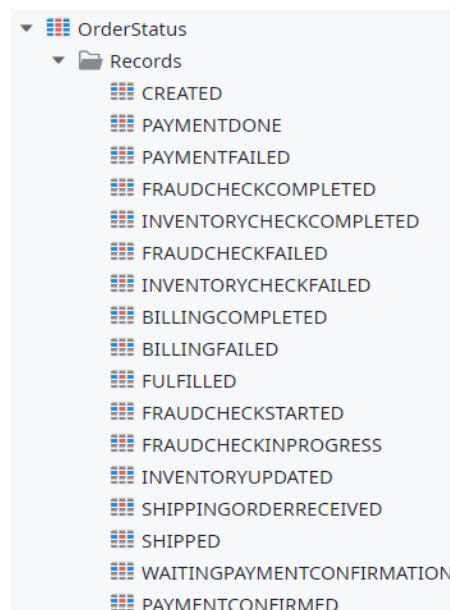
The app has several important Server Actions that support the order workflow. When the user clicks the Purchase Button, the **StartPurchase** Action is called.

- **StartPurchase:** This Action has the functionality that supports the entire sales process. It begins by saving the order, checking the inventory, doing the payment and verifying if there is any fraud, updating the inventory, shipping the order, and handling the billing. Between every step, we update the order with the respective status.
- **SaveOrder:** This Action creates an Order and adds the items in the cart to the order.
- **FraudDetectionCheck:** The Action simulates the fraud verification. The process involves a Sleep Action to mimic a delay between verifications. Subsequently, the order undergoes multiple updates to reflect its changing status as fraud detection progresses to completion. Please note that there is no logic to verify the existence of fraud.
- **InventoryCheck, PickInventory:** These Actions simulate the verification of whether there's inventory available for the order and then update the order accordingly.
- **DoShipping:** This Action updates the order by simulating its shipping.
- **DoBilling:** This Action simulates the payment and billing processing.
- **CreateOrUpdateOrderWrapper:** This Action encapsulates the logic to update the Order and its Status throughout the process.

To enable this functionality, the data model includes the Order and OrderStatus entities, which are frequently utilized throughout the logic.



The Order Entity has several Status fields that are updated throughout the workflow, with text fields indicating the status of the payment, shipping, etc. The Status field links this Entity with the OrderStatus Entity that has several states of the workflow, as the next image indicates:



Hands-on

The purpose of this exercise is to improve the existing app by having this process loosely coupled, leveraging Events and Service Actions. This exercise must be done individually and consists of two parts: designing the workflow for Events and enhancing the existing app in ODC Studio.

Requirements

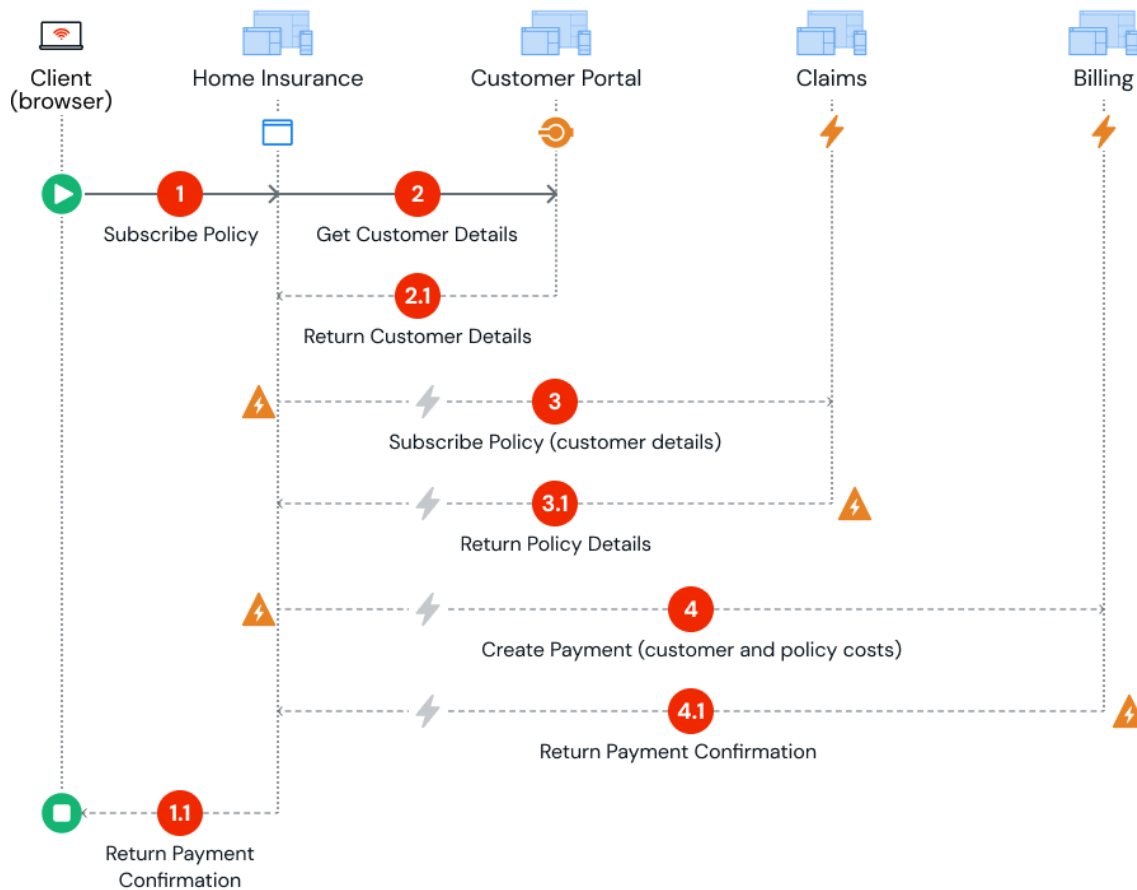
EDA has hired you to design a solution that requires minimal code changes, is scalable, cost-effective, and supports parallelization. Keep the following points in mind while designing:

1. The design of the solution should be loosely coupled, based on Events and Service Actions to support the several steps of the workflow. For instance, when the payment is done, an Event should be triggered to activate the fraud checking process, and so on.
2. Support for notifications via SMS, WhatsApp, and Email. Notifications should be sent upon the completion of each phase of the process.
3. Each stage of the workflow process should have a failure mechanism.

Create a diagram with all the process steps

This order process consists of several steps, beginning with a new order and continuing through to shipping and billing. Now, all these steps are implemented in a single app. To effectively use the loosely coupled nature of events, divide the steps among different applications to manage each business context.

Before accessing the app, we must first create the ODC blueprint detailing the flow of events and transactions. You can use the following image as an example:



Don't forget to represent the Events you believe are important to support the several stages of the EDA sales process.

This diagram should be the foundation for creating the apps, Events, and Actions in the next part of the exercise. You can use any tool you like to design the diagram, but try to use a tool that makes it easy to share with others. If you are on location, any whiteboard or paper sheet works!

Improve the app

Now that we have built the diagram, it's time to improve the existing app. Open ODC Studio and analyze the quickstart app. Start by splitting the app into multiple apps. This should follow the diagram you built before and should be consistent with the several steps of the workflow.

Then, think about the number of Events you will need and place them in the respective app. The final step is to adjust the Actions in the existing app to start using the Events and separate them through the respective apps.

You can follow the following suggestions:

- Split the Actions of the current app into new Actions in the respective application.
- Create simple apps highlighting key implementation aspects based on the existing logic, and fill any gaps that the logic may have.
- In case of calls to external systems to process payment, detect fraud, or send notifications, do not worry about the implementation. You can create empty wrappers and use Messages to indicate the notification has been sent or the call has been completed.